BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Multi-select

arxiv.org

https://arxiv.org/pdf/1810.04805

[개념 정리]

<(파이토치 트랜스포머를 활용한) 자연어 처리와 컴퓨터 비전 심층학습>

- BERT (p.406~416)

📒 BERT

1. **BERT**

   2018년 구글에서 발표한 언어모델, 트랜스포머 기반 양방향 인코더를 사용하는 자연어 처리 모델

   - **양방향(Bidirectional) 인코더**: 입력 시퀀스를 양쪽 방향에서 처리해 이전과 이후의 단어를 모두 참조하면서 단어의 의미와 문맥을 파악함

     ⇒ 기존 모델들보다 더 정확하게 문맥을 파악하고, 다양한 자연어 처리 작업에서 높은 성능
   - 대규모 데이터를 사용해 사전학습 되어있음 → 전이 학습에 주로 활용
   - 일부 또는 전체를 다른 작업에 재사용, 적은 양의 데이터로도 높은 정확도 & 시간 단축
   - 트랜스포머의 인코더 모듈만 사용해 입력 문장 처리
     - **인코더:** 입력 문장의 단어들을 임베딩 (각 단어의 의미 벡터화) → 순차적으로 처리하여 문장 전체의 의미 추출
     - 인코더만 사용해 입력 문장의 단어를 좌우 방향으로 처리 → 문맥 정보 모델링

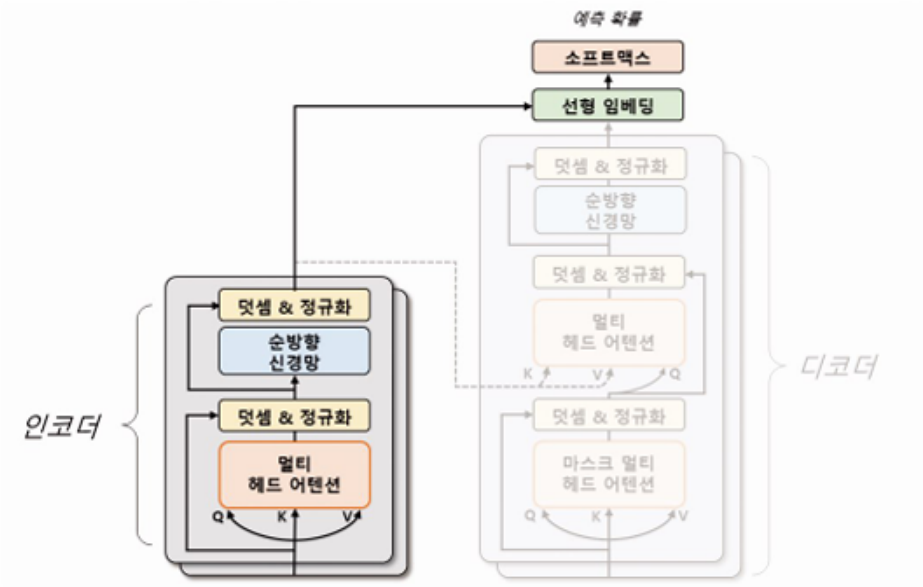


그림 7.13 트랜스포머와 BERT 모델 비교

2. **사전 학습 방법**

- 마스킹된 언어 모델링(Masked Language Modeling, MLM)
  : 입력 문장에서 임의로 일부 단어 마스킹하고 해당 단어 예측하는 방식
  - I'm [learning → MASK] Pytorch 에서 MASK 단어 예측
  - 양방향 문맥 정보를 참고하여 누락 단어 추론함
  - 이 학습방법으로 MERT의 문장 전체의미 이해능력 향상
- 다음 문장 예측(Next Sentence Prediction, NSP)
  : 두 문장이 있을 때, 두 번째 문장이 첫 번째 문장 다음 문장인지 여부 판단
  - 두 문장의 연속 관계를 예측
  - 문장 간 관계 학습 + 문장 간 의미적 유사성 파악
- 입력 문장에 특수 토큰 추가
  - **[CLS] 토큰**: 입력 문장의 시작 부분에 추가. 문장 분류 작업을 위한 정보제공. 입력 문장이 어떤 유형(긍정/부정 or 어떤 객체가 언급되는지 분류)인지 미리 파악가능
  - **[SEP] 토큰**: 입력 문장 내에서 두 개 이상의 문장을 구분하기 위해 사용. 두 개의 독립적인 문장을 인식하고 각 문장의 정보를 정확히 파악
  - **[MASK] 토큰**: 입력 문장 내에서 임의로 선택된 단어를 가리키는 토큰. 일부 단어를 가려 모델의 학습과 예측에 활용함

        - 텍스트 토큰 중 15% 단어를 대상으로 마스킹 - 80%는 [MASK] 토큰으로 + 10%는 어휘사전의 무작위 단어로 변경 + 10% 실제 토큰 그대로
    - 입력 시퀀스: [CLS] 문장1 [SEP] 문장2 [MASK] [SEP] ...
- BERT: 사전학습 → 미세 조정 과정 (해당 작업에 맞는 계층 추가, 손실 함수 정의 후 학습) ⇒ 다양한 자연어 처리 적용(문장 분류, 감성 분석, 질문 응답, 기계 번역 등)

3. 모델 실습

📄 from transformers import BertTokenizer

- 데이터: 네이버 영화리뷰 감정 분석 데이터세트
    - 20,000개의 데이터세트 → 6:2:2 = 학습:검증:테스트 데이터
- BERT를 활용한 분류 모델 학습
    - BertTokenizer로 사전학습된 토크나이저(다중 언어 지원, 대소문자 유지)를 불러와 전처리 수행
        - 소문자 유지(do_lower_case+True) → Apple ≠ apple 다르게 인식
    - 데이터 토큰화, 데이터세트로 변경(숫자 ID, 어텐션 마스크, 레이블은 텐서형식), 데이터로더 적용, get_dataloader 함수로 샘플링, 모델과 최적화 함수 선언
    - RandomSampler: 데이터 무작위 샘플링 → 학습에 적용
      SequentialSampler: 데이터를 고정된 순서대로 반환 → 검증 및 평가 배치에 적용

📄 from transformers import BertForSequenceClassification

```
••• bert
    L embeddings
    | L word_embeddings
    | L position_embeddings
    | L token_type_embeddings
    | L LayerNorm
    | L dropout
    L encoder
    | L layer
    | | L 0
    | | L 1
    | | L 2
    | | L 3
    | | L 4
    | | L 5
    | | L 6
    | | L 7
    | | L 8
    | | L 9
    | | L 10
    | | L 11
    L pooler
    | L dense
    | L activation
    dropout
    classifier
```

- 모델선언: BertForSequenceClassification.from_pr
    - 패딩 기법 사용하므로 모델 설정 변경X
- AdamW 활성화 함수 사용: Adam+가중치 감쇠 추가
    - 학습률을 0으로 나누는 것을 방지하기 위해 분모에 eps를 더함
    - 모든 매개변수의 학습률을 조절하는 적응형 활성화 함수 → 모델 빠르게 수렴, 안정적 학습 (대규모 매개변수 딥러닝 모델이므로 안정적인 기울기 갱신이 필수)

```
Epoch 1: Train Loss: 0.2871 Val Loss: 0.4581 Val Accuracy0.8130
Saved the model weights
Epoch 2: Train Loss: 0.2216 Val Loss: 0.5109 Val Accuracy0.8147
Epoch 3: Train Loss: 0.1869 Val Loss: 0.5684 Val Accuracy0.8073
Epoch 4: Train Loss: 0.1576 Val Loss: 0.6563 Val Accuracy0.7880
Epoch 5: Train Loss: 0.1207 Val Loss: 0.6401 Val Accuracy0.8150
```

```
Loading weights: 100%|▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 199/199 [00:00<00:00, 672.14it/s, Materializing param=
BertForSequenceClassification LOAD REPORT from: bert-base-multilingual-cased
Key                                          | Status     |
---------------------------------------------+------------+-
cls.predictions.transform.dense.bias         | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias     | UNEXPECTED |
cls.predictions.transform.dense.weight       | UNEXPECTED |
cls.seq_relationship.bias                    | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight   | UNEXPECTED |
cls.seq_relationship.weight                  | UNEXPECTED |
cls.predictions.bias                         | UNEXPECTED |
classifier.bias                              | MISSING    |
classifier.weight                            | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture
- MISSING   :those params were newly initialized because missing from the chec
Test Loss: 0.4712
Test Accuracy: 0.8033
```

- 작은 데이터세트에서도 일반화 가능함
- 실습 코드

Google Colab

https://colab.research.google.com/drive/1Lunb6PK494MrbRCoSFfwlIDH3Sk4KhSN?usp=sharing

---

[논문 정리]

📗 논문 링크: BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

1. Introduction

논문이 다루는 분야

자연어 이해(Language Understanding)를 위한 **언어 표현 모델의 사전 학습(Pre-training)**분야. 문장 전체를 분석하여 관계를 예측하는 문장 수준의 작업과 단어 단위의 정밀한 출력이 필요한 토큰 수준의 작업을 모두 아우르는 범용적인 언어 모델 구축

해당 task에서 기존 연구 한계점

자연어처리에서 사전학습 표현을 다운스트림 작업에 적용하기 위한 기존 전략

- **Fine-tuning (ex. OpenAI GPT)**
  단방향(왼-오, 오-왼) 언어 모델을 사용하여 학습함 (이전/이후 토큰 중 하나에만 주목)

  → 문맥 이해 부족 (특히 Q&A 같이 양방향 문맥이 필수적인 토큰 작업에 적용될 때 모델 성능 저하)

- **Feature-based (ex. ELMo)**
  왼쪽-오른쪽, 오른쪽-왼쪽으로 학습된 두 모델을 단순히 결합(Concatenation)한 형태

  → 모든 레이어에서 양방향 문맥 동시 고려 X

기존 전략 비교 분석 (ELMo vs GPT)

비교 항목	ELMo (Feature-based)	OpenAI GPT (Fine-tuning)
아키텍처	LSTM 기반	트랜스포머 디코더 기반
방향성	얕은 양방향 (순방향/역방향의 단순 결합)	단방향 (Left-to-Right)
적용 방식	사전 학습된 벡터를 고정된 특징으로 사용	모든 파라미터를 하위 과제에 맞춰 업데이트
한계점	작업별 복잡한 아키텍처 설계 필요	단방향성으로 인한 양방향 문맥 파악의 한계

논문의 contributions

- 모든 레이어에서 왼쪽과 오른쪽 문맥을 **동시에** 고려하여 학습하는 **BERT(Bidirectional Encoder Representations from Transformers)** 모델을 제안하여 기존의 단방향성 제약 극복

  → 사전학습된 BERT 모델은 최소한의 아키텍처 수정으로 fine-tuning 가능 ⇒ 범용성!
  - 특정 작업을 위해 고도로 설계된 복잡한 아키텍처의 필요 X

- **새로운 사전 학습 목적 함수**
  - **마스크드 언어 모델(MLM):** 입력 토큰 일부를 무작위로 가리고 주변 문맥만으로 원래 단어를 예측 → 깊은 양방향 표현 학습
  - **다음 문장 예측(NSP):** 두 문장 간의 관계를 이해하도록 학습

- GLUE, SQuAD v1.1 및 v2.0을 포함한 11개의 주요 NLP 작업에서 최첨단 성능 기록

2. Related Work

언어 표현의 사전 학습 방법들

1. **Unsupervised Feature-based Approaches)**
   - 수십 년간 연구되어 온 분야, 비신경망과 신경망 방법 모두 포함
   - 사전 학습된 단어 임베딩은 NLP의 필수요소, 처음부터 학습하는 것보다 성능 크게 개선

- 학습 목적 함수: 단어 벡터 사전 학습을 위해 왼쪽→오른쪽 언어 모델링이나, 문맥 속에서 올바른 단어랑 못된 단어를 구별하는 방식 등 사용
    - → 문장/문단 임베딩 같이 큰 단위로 일반화
    - 다음 문장의 순위를 매기거나 노이즈제거하는 오토인코더 방식을 학습에 사용
- ELMo: 이 연구를 문맥에 민감한 특징 추출 단계로 발전시킴

2. **Unsupervised Fine-tuning Approaches**
    - Feature-based와 비슷하게 시작해 더 포괄적인 방식으로 발전
    - 인코더 사전학습: 문맥적 토큰 표현을 생성하는 문장/문서 인코더를 라벨이 없는 텍스트에서 사전학습, 이를 실제 작업을 위해 미세 조정
    - 처음부터 학습해야하는 매개변수가 거의 없는 장점
    - OpenAI GPT: 왼쪽→오른쪽 언어 모델링이나 오토인코더 목적함수를 학습에 사용
3. **Transfer Learning from Supervised Data**
    - 충분한 데이터가 있는 지도학습 연구도 진행
    - 자연어 추론(NLI)이나 기계 번역 같이 대규모 데이터셋이 있는 지도학습에서 얻은 모델을 다른 작업으로 옮겼을때 효과적
    - 컴퓨터 비전에서도 ImageNet 사전학습된 모델을 미세 조정해서 사용한 사례

3. 제안 방법론

Main Idea

✅ **BERT Architecture**

: 사전 학습(Pre-training)+파인 튜닝(Fine-tuning)

- 사전학습: 모델은 라벨이 없는 데이터로 학습 (통합 구조)
- 파인튜닝: 사전 학습된 파라미터로 초기화 + 실제(downstream) task의 데이터를 사용해 모든 파타미터 미세 튜닝

- 사전 모델: 다층 양방향 트랜스포머 인코더를 기반으로 함
- 두 가지 모델 크기를 중점으로 다룸 (레이어 수 (즉 Transformer 블록 수): L / 은닉층 크기: H / 셀프 어텐션 헤드 수: A)
    - $BERT_{BASE}$:L=12,H=768,A=12, 피드포워드/필터 크기=3072, 전체 파라미터 110M
    - $BERT_{LARGE}$: L=24,H=1024,A=16, 피드포워드/필터 크기=4096, 전체 파라미터 340M

- $BERT_{BASE}$는 OpenAI GPT와의 비교를 위해 동일 모델 크기로 설계 (양방향 셀프어텐션 / 제한된 셀프 어텐션 차이)

✅ **Input/Output Representations**

: BERT가 다양한 다운스트림 태스크를 처리할 수 있도록 입력 표현은 단일문장과 문장 쌍을 하나의 토큰 시퀀스로 함. 논문에서는 30,000개 단어 규모의 WordPiece 임베딩 사용

- 다른 두 문장을 하나의 시퀀스 안에 이어 붙여 입력 → 문장 간의 관계 파악 가능 (ex. 질문 답변, 자연어추론, 다음문장 예측)
- [CLS]: 시퀀스의 첫 토큰, [SEP]: 문장 구분, 세그먼트 임베딩: 토큰이 문장 A or B에 속하는지 나타냄
  - 최종 입력 표현: 1+2+3
    1. **Token Embeddings**: WordPiece 단위의 벡터 표현
    2. **Segment Embeddings**: 문장 A/B 구분 벡터
    3. **Position Embeddings**: 시퀀스 내 위치 정보 벡터

특수 토큰의 역할 정리

토큰	역할	비고
[CLS]	시퀀스 시작점에 위치하며, 전체 문맥을 응축하여 분류용 벡터로 사용됨	모든 입력에 필수
[SEP]	두 문장 사이를 구분하거나 시퀀스의 끝을 나타냄	문장 쌍 작업 시 사용
[MASK]	사전 학습 시 모델이 예측해야 할 대상 토큰을 가림	**사전 학습 시에만 사용됨**

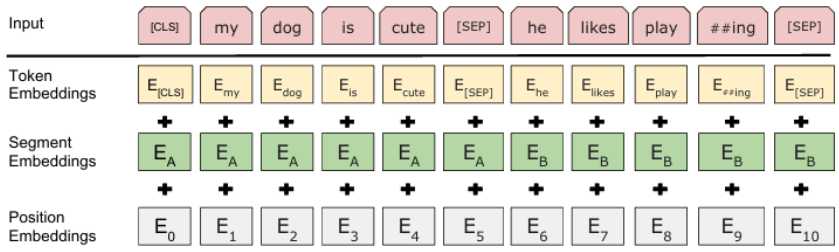


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

3.1 Pre-training BERT (사전학습 과제)

기존의 단방향 언어 모델과 달리 두 가지 비지도 학습 task로 깊은 양방향 표현 학습

Task #1: Masked LM(MLM)

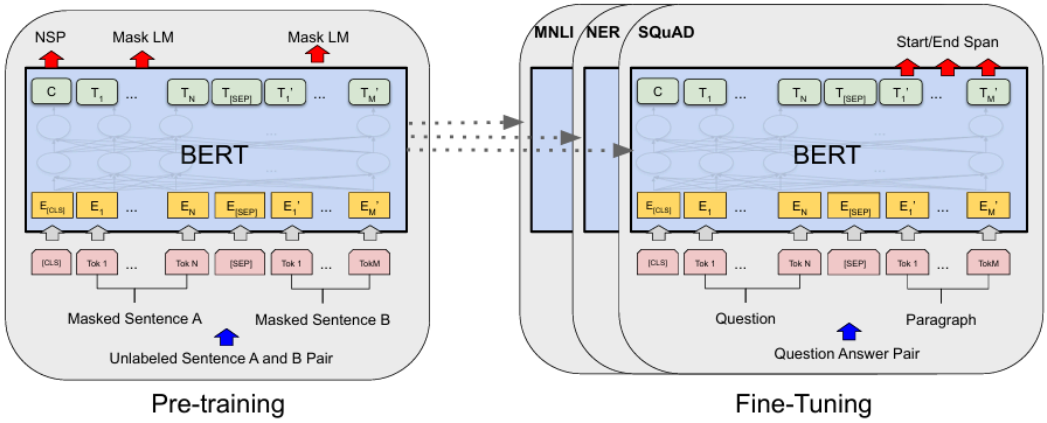
- 깊은 양방향 표현 학습을 위해 입력 토큰의 15%를 무작위로 선택해 마스킹 (Cloze 태스크)
- Fine Tuning시엔 [MASK]토큰이 나타나지 않으므로, 15% 토큰 중
  - 80% → [MASK] **my dog is [MASK]**
  - 10% → 무작위 단어 **my dog is apple**
  - 10% → 변경X **my dog is hairy**

Task #2: Next Sentence Prediction(NSP)

- 문장 간의 관계를 이해해야하는 작업을 위해 (Q&A, NLI...) 문장 B가 문장 A 음에 오는 문장인지 (IsNext) 아닌지(NotNext)를 예측하는 이진분류 태스크 수행
- 데이터의 50%는 실제 후속 문장, 50%는 무작위 문장으로 구성

3.2 Fine-Tuning BERT

- 트랜스포머의 셀프 어텐션 메커니즘 → 입력을 적절히 교체하여 다양한 tzsk를 모델링 가능
- 기존: 텍스트 쌍을 독립적으로 인코딩한 후 교차 어텐션을 적용
  BERT: 두 문장을 단일 시퀀스로 결합해 셀프 어텐션 수행 → 문장 간 양방향 교차 어텐션 자연스럽게 포함
  - 분류 task: [CLS] 토큰의 최종 은닉 상태 C를 출력 레이어에 입력
  - 토큰 수준 task(NER, Q&A): 각 토큰의 최종 은닉 벡터 T_i를 출력 레이어에 입력해 개별 라벨이나 정답 구간의 시작/종료 지점 예측



4. 실험 및 결과

Dataset

4.0 사전학습 Pre-training BERT

▼ Dataset: BooksCorpus(8억 단어) 및 English Wikipedia(25억 단어, 텍스트만 추출)

선정 이유: 문서 수준의 말뭉치 확보

- **연속된 긴 문맥 학습:** 'Billion Word Benchmark'와 같이 문장이 무작위로 섞인 데이터셋과 달리, 위 데이터들은 길고 연속된 텍스트 시퀀스를 추출하는 데 유리
- **NSP 작업 수행:** 다음 문장 예측을 위해, 두 문장이 실제로 이어지는지 판단해야 하므로 문서 단위의 일관된 흐름이 있는 데이터 필수
- **Wikipedia 전처리:** Wikipedia의 경우 리스트, 표, 헤더 등은 무시하고 순수 텍스트 passages만 추출해 학습의 효율성 높임

- **하이퍼파라미터**: 배치 크기 256 sequences, 총 1,000,000 steps 학습
  - 학습률 1e-4, Adam 옵티마이저($\beta 1$=0.9,$\beta 2$=0.999), L2 가중치 감쇠 0.01
- 학습 효율을 위해 전체 스텝의 90%는 시퀀스 길이 128로 학습, 10%는 위치 임베딩 학습을 위해 시퀀스 길이 512로 학습

4.1 GLUE (General Language Understanding Evaluation)

- **Dataset:** 다양한 자연어 이해 작업을 모은 벤치마크
  - MNLI(자연어 추론), QQP(질문 쌍 유사도), QNLI(질문-문장 추론), SST-2(감성 분석), CoLA(언어 용인성), STS-B(의미적 유사성), MRPC(패러프레이징), RTE(문장 함의) 등 포함
- **Baseline:** 기존 SOTA(최첨단) 모델들, BiLSTM+ELMo+Attn, OpenAI GPT 사용
- **결과:** *BERTBASE*와 *BERTLARGE* 모두 기존 모델들을 큰 차이로 능가함.
  - 특히 *BERTLARGE*는 GLUE 평균 점수 80.5를 기록하여 OpenAI GPT(72.8) 대비 압도적인 성능 향상 보임

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	**Average** -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

4.2 SQuAD v1.1 (Stanford Question Answering Dataset)

- **Dataset:** 10만 개의 질문/답변 쌍으로 구성된 데이터셋. 주어진 지문에서 질문에 대한 답에 해당하는 텍스트 범위를 찾는 작업

- **Baseline:** 인간의 성능(Human), 당시 리더보드 상위 시스템(nlnet, QANet), 기존 모델들(BiDAF+ELMo 등)과 비교
- **결과:** BERT 앙상블 모델은 리더보드 1위 시스템보다 F1 점수 기준 +1.5 더 높은 성적을 거두었고 단일 모델로도 기존의 모든 앙상블 시스템 능가

System	Dev		Test	
	EM	F1	EM	F1
Top Leaderboard Systems (Dec 10th, 2018)				
Human	-	-	82.3	91.2
#1 Ensemble - nlnet	-	-	86.0	91.7
#2 Ensemble - QANet	-	-	84.5	90.5
Published				
BiDAF+ELMo (Single)	-	85.6	-	85.8
R.M. Reader (Ensemble)	81.2	87.9	82.3	88.5
Ours				
$\text{BERT}_{\text{BASE}}$ (Single)	80.8	88.5	-	-
$\text{BERT}_{\text{LARGE}}$ (Single)	84.1	90.9	-	-
$\text{BERT}_{\text{LARGE}}$ (Ensemble)	85.8	91.8	-	-
$\text{BERT}_{\text{LARGE}}$ (Sgl.+TriviaQA)	**84.2**	**91.1**	**85.1**	**91.8**
$\text{BERT}_{\text{LARGE}}$ (Ens.+TriviaQA)	**86.2**	**92.2**	**87.4**	**93.2**

4.3 SQuAD v2.0

- **Dataset:** SQuAD 1.1에 '답이 없는 경우'를 추가해 더 현실적으로 만든 데이터셋
- **Baseline:** MIR-MRC, nlnet, unet, SLQA+ 모델
- **결과:** 이전 최고 시스템 대비 +5.1 F1 점수 향상

System	Dev		Test	
	EM	F1	EM	F1
Top Leaderboard Systems (Dec 10th, 2018)				
Human	86.3	89.0	86.9	89.5
#1 Single - MIR-MRC (F-Net)	-	-	74.8	78.0
#2 Single - nlnet	-	-	74.2	77.1
Published				
unet (Ensemble)	-	-	71.4	74.9
SLQA+ (Single)	-		71.4	74.4
Ours				
$\text{BERT}_{\text{LARGE}}$ (Single)	78.7	81.9	80.0	83.1

4.4 SWAG (Situations With Adversarial Generations)

- **Dataset:** 11.3만 개의 문장 쌍 완성 예제로 구성되어 상식적인 추론 능력평가. 네 가지 선택지 중 가장 그럴듯한 이어지는 문장을 고르는 작업
- **Baseline:** 기준 모델 ESIM+ELMo와 OpenAI GPT 사용
- **결과:** *BERTLARGE*는 ESIM+ELMo보다 27.1%, OpenAI GPT보다 8.3% 더 높은 정확도 기록

System	Dev	Test
ESIM+GloVe	51.9	52.7
ESIM+ELMo	59.1	59.2
OpenAI GPT	-	78.0
$\text{BERT}_{\text{BASE}}$	81.6	-
$\text{BERT}_{\text{LARGE}}$	**86.6**	**86.3**
Human (expert)†	-	85.0
Human (5 annotations)†	-	88.0

4.5 Ablation Studies

: BERT 모델의 특정 구성 요소나 기능을 하나씩 제거하거나 수정해보며 그 요소가 전체 성능에 얼마나 기여하는지 확인

- **4.5.1 Effect of Pre-training Tasks:**
  - **실험:** 다음 문장 예측(NSP) 작업 빼보기(No NSP) / 마스크드 언어 모델(MLM) 대신 단방향(LTR)방식으로만 학습
  - **결과:** no NSP→ SQuAD이나 추론 성능 저하, 단방향 학습 시 모든 작업에서 양방향 모델보다 훨씬 낮은 성능
- **4.5.2 Effect of Model Size:**
  - **실험:** 레이어 수, 숨겨진 유닛 수, 어텐션 헤드 수를 다양하게 바꿔가며 모델 크기 키우기
  - **결과:** 모델이 커질수록 성능이 지속적으로 향상. 학습 데이터가 아주 적을 때도 모델 크기를 키우면 성능이 크게 향상
- **4.5.3 Feature-based Approach with BERT:**
  - **실험:** BERT를 Fine-tuning하지 않고, 고정된 Feature extractor (예: ELMo 방식)로만 사용했을 때 성능 측정
  - **결과:** 특징만 추출해서 사용해도 성능이 상당히 우수함. BERT가 Fine-tuning 방식뿐만 아니라 Feature기반 방식에서도 매우 효과적이기 때문

⇒ BERT의 성능 향상은 단순히 데이터 양 때문이 아니라 깊은 양방향 구조 + 설계된 학습 작업들의 유기적인 결합 덕분

결과

- 사전 학습 방식 덕분에 학습 데이터가 부족한 저자원 작업도 깊은 단방향 구조의 이점을 누릴 수 있었음

  → 기존의 성과를 깊은 양방향 구조(deep bidirectional architectures)로 확장

⇒ 동일한 사전 학습 모델 하나만으로도 광범위한 NLP 작업들을 성공적으로 해결

5. 결론 (배운점)

자연어를 이해할때는 양방향의 문맥 이해가 필수적인데, 기존 연구의 한계를 극복하고 양방향 트랜스포머를 이용해 성공적으로 구현해낸 점에 큰 의의가 있다. 사전학습된 표현을 이용해 조금씩 튜닝하여 범용성을 갖춘 점이나 매우 적은 데이터로도 모델의 크기를 키워 성능향상에 기여할 수 있는 점도 자연어 이해 작업의 성능을 획기적으로 높이는데 기여하였다. 컴퓨터가 문맥을 유기적으로 학습할 수 있게 되면서 인간의 언어를 정교하게 이해할 수 있는 토대가 된 중요한 연구인 것 같다.

또한 Ablation Study들로 모델의 특정 요소가 성능에 얼마나 기여하는지 알아보는 실험을 통해 핵심 요소의 중요성을 강조한 흐름이 인상깊었다.